

01 / Alcance y arquitectura

Objetivo de la rama

La rama `comp_portafolio` parte de `primeros_pasos`, commit `ac4a73f`. Se desarrolla Compra y se extrae Portfolio como componente formal. Se añade un simulador pequeño para evidenciar estado conversacional. La entrega general requiere Compra, Venta y Portfolio; Venta no se declara terminada ni se reemplaza por el simulador. El equipo está desarrollando pantallas de inicio de sesión y base de datos en trabajos todavía no publicados.

Separación de responsabilidades

La presentación está formada por JSP, JavaScript, Servlet y recursos JAX-RS. Recibe peticiones y entrega DTOs. La capa de negocio contiene interfaces locales e implementaciones EJB, validaciones, cálculo y coordinación transaccional. La capa de datos conserva las entidades y los repositories JPA existentes. El servidor administra las dependencias y transacciones; no se instancian servicios EJB con `new` en producción.

Recorrido de una compra

`POST /api/compras` atraviesa la seguridad web y `CompraResource`. `CompraService` valida el instrumento, cantidad, precio y fecha; resuelve la cartera de la identidad activa mediante `PortfolioActual` y persiste una Operación. La respuesta 201 obtiene el resumen desde `PortfolioService`. El dashboard usa ese mismo contrato para actualizar sus importes. El total se calcula en el servidor y no se confía en un importe enviado por el navegador.

Componentes y dependencias

Compra: `REST/dashboard` -> `CompraService` -> `InstrumentoRepository`, `PortfolioActual` y `OperacionRepository`.
Portfolio: `REST/dashboard` -> `PortfolioService` -> `PortfolioActual`, `OperacionRepository` y `CotizacionStrategy`.
Simulador: `REST` -> `SimulacionSesion` -> `SimuladorPortfolioService`. El simulador no incorpora DAO porque no guarda inversiones ni necesita base de datos. El estado HTTP solo contiene una referencia al EJB conversacional y su identidad propietaria.

Límites expresos

No se desarrollan Venta, pantallas de login, importación de precios históricos, pagos, billeteras, rentabilidad histórica ni nivel de riesgo. Los instrumentos conservan cotizaciones ilustrativas del catálogo. Todos los importes de la demo se interpretan en USD; una integración multimonedada debe convertir antes de sumar patrimonio. No se afirma que los precios sean cotizaciones actuales de mercado.

02 / Compra, transacciones y reglas

Contrato de negocio

CompraService expone listarInstrumentos() y registrarCompra(CompraRequest). Se retiró obtenerResumen() para que Compra no consolide posiciones. CompraRequest contiene ticker, cantidad, precioUnitario y fecha. El precio real de una operación puede diferir de la cotización del catálogo. El servicio normaliza el ticker con trim y Locale.ROOT y comprueba que exista.

Validaciones de entrada

Cantidad y precio deben ser positivos. La fecha es obligatoria y no puede ser futura. Se respetan las precisiones de persistencia: cantidad con hasta seis decimales y trece enteros; precio con cuatro decimales y quince enteros. El total se redondea a cuatro decimales HALF_UP y debe caber en la columna decimal(19,4). Un total inferior a la precisión monetaria se rechaza. Entradas nulas, ticker vacío o inexistente producen ReglaNegocioException.

Consistencia transaccional

registrarCompra utiliza TransactionAttribute REQUIRED: se une a una transacción activa o el contenedor crea una. Los repositories participan en esa transacción. ReglaNegocioException es ApplicationException con rollback=true, por lo que una regla fallida cancela los cambios asociados. Operacion calcula su total a partir de cantidad y precio y conserva el tipo COMPRA. No se gestionan conexiones ni commits manualmente.

Ciclo de vida stateless

CompraServiceBean está anotado Stateless y registra mensajes INFO desde PostConstruct y PreDestroy. El contenedor crea instancias según demanda y puede reutilizarlas entre usuarios. Los campos solo contienen dependencias y un logger; la identidad y la operación viven en variables de cada invocación. PostConstruct no ocurre en cada compra, y PreDestroy normalmente se observa al retirar el despliegue. Los logs constituyen evidencia del servidor, no de un objeto construido en una prueba unitaria.

Errores y alcance transaccional

Los errores de negocio se traducen a HTTP 400 con mensaje JSON. La restricción web devuelve 401/403 antes de ejecutar la operación si falta identidad o rol. La escritura de Compra y la consulta de resumen son llamadas de servicio distintas: una falla posterior de lectura no revierte una compra ya confirmada. El producto definitivo debería incorporar una clave de idempotencia para reintentos; esta rama no implementa pagos ni ejecuciones de mercado.

03 / Portfolio y cálculo financiero

Servicio formal

PortfolioService es una interfaz EJB local; PortfolioServiceBean es Stateless y una Service Facade. Expone obtenerResumen() y obtenerPosiciones(). El resource usa GET /api/portfolio/resumen y GET /api/portfolio/posiciones. No recibe IDs de carteras arbitrarias. Consulta las operaciones de la cartera resuelta por identidad y construye DTOs, sin devolver entidades administradas ni lógica JPA a la interfaz.

Fórmulas

Por ticker, cantidad = suma de cantidades; capital invertido = suma de totales registrados; precio promedio = capital invertido / cantidad, con cuatro decimales HALF_UP. Valor actual = cantidad por cotización de Strategy, redondeado a cuatro decimales. Ganancia no realizada = valor actual menos capital invertido. Rendimiento porcentual = ganancia por 100 / capital invertido. Para base cero se devuelve cero; el portfolio vacío entrega lista vacía e importes cero.

Ejemplo verificable

Comprar diez AAPL a USD 180 y cinco a USD 210 produce quince unidades y USD 2.850 invertidos. El precio promedio es USD 190. Con cotización de USD 195, el patrimonio vale USD 2.925, la ganancia es USD 75 y el rendimiento 2,6316%. Las pruebas comprueban este promedio ponderado, además de una posición con pérdida y una cartera vacía. No se promedian sin ponderar los precios de ambas compras.

Contratos de salida

PosicionDto conserva nombre, ticker, tipo, cantidad, precioPromedio, invertido, actual y rendimiento. Se añaden precioActual y rendimientoPorcentaje. rendimiento mantiene su significado previo de ganancia monetaria para compatibilidad con el dashboard. ResumenPortfolioDto contiene posiciones, capitalInvertido, patrimonioTotal, gananciaTotal y rendimientoPorcentaje. Las posiciones se ordenan por nombre y los totales se agregan de forma determinista.

Integración de Venta

El cálculo actual solo acepta COMPRA y falla ante otro tipo; es una protección contra sumar ventas como adquisiciones. Al integrar Venta se acordará costo promedio móvil o la política elegida, se procesarán movimientos cronológicamente y se distinguirá ganancia realizada de no realizada. Venta deberá reutilizar PortfolioActual y repositories, validar stock y resolver concurrencia. No se agregaron endpoints o contratos ficticios de Venta. El frontend antiguo que usa /portfolios/1/resumen debe migrar a /portfolio/resumen.

04 / Stateful y patrones de diseño

Conversación del simulador

SimuladorPortfolioServiceBean es Stateful y guarda capital y porcentajes por TipoInstrumento entre peticiones. SimulacionSesion es CDI SessionScoped, serializable, y recibe una referencia EJB exclusiva de la sesión HTTP. PUT capital define el importe; PUT porcentajes/{tipo} cambia una asignación; GET obtiene importes y saldo disponible. El cliente debe conservar JSESSIONID. La sesión también se vincula a la identidad que la inició para impedir que otra identidad consulte la simulación reutilizando la cookie.

Reglas y cierre

El capital no puede ser negativo, exceder 999999999999,99 ni tener más de dos decimales. Los porcentajes están entre cero y cien, admiten hasta cuatro decimales y su suma no supera cien. Se permiten distribuciones parciales y se informa disponible. Importe = capital por porcentaje / 100; se conservan fracciones de centavo para no redistribuir por redondeo. Se devuelven copias de los mapas. DELETE invalida la sesión, invoca Remove y dispara PreDestroy. El timeout HTTP es de treinta minutos.

Evidencia del contenedor

PostConstruct y PreDestroy del simulador registran un identificador de conversación. Se pueden mostrar dos sesiones con identificadores distintos y seguir el mismo ID hasta su destrucción. El EJB no utiliza JPA ni transacciones de BD: NOT_SUPPORTED expresa esa decisión. cerrar tiene PermitAll para liberar recursos al terminar la autenticación; no se expone como API pública, pues todo /api permanece protegido. La diferencia con stateless es conservar estado por conversación, no el hecho de tener o no una base de datos.

Patrón 1: Repository / DAO

InstrumentoRepository, PortfolioRepository y OperacionRepository encapsulan EntityManager y consultas. Compra y Portfolio solicitan operaciones del dominio sin construir SQL ni consultas desde la presentación. Se reutiliza la persistencia existente, evitando un repositorio paralelo para el resumen. Repository/DAO cuenta como un patrón, no dos.

Patrones 2 y 3: Facade y Strategy

PortfolioService es una Service Facade porque coordina identidad, consulta, agrupación y cotización detrás de un contrato pequeño de resumen. CotizacionStrategy define cotizar(Instrumento); CotizacionCatalogo es la implementación CDI activa. El algoritmo de agregación depende de la interfaz y una prueba sustituye la cotización. Esto permite integrar otro origen sin cambiar la consolidación. Al añadir implementaciones, usar qualifiers o una alternativa CDI seleccionada para evitar inyección ambigua. No se alegan Factory o Adapter inexistentes.

05 / Seguridad, persistencia e integración

Autenticación y autorización

web.xml protege /api/* y /dashboard, usa BASIC en ApplicationRealm y declara USUARIO. jboss-web.xml selecciona other, configuración de seguridad estándar de WildFly. Los EJB de Compra, Portfolio y resolución de identidad exigen RolesAllowed USUARIO. La identidad llega desde el servidor; no se acepta un usuario enviado por JSON ni se usa RunAs para elevar permisos. Para la demo se crean Application Users con el asistente add-user y el grupo USUARIO. Fuera de localhost es necesario HTTPS.

Propiedad de la cartera

PortfolioActualBean obtiene el principal de SessionContext, verifica el rol y busca su propietario en PortfolioRepository. Si no existe, crea una cartera para ese principal. La columna propietario es un puente de integración con unicidad; no reemplaza el diseño Usuario-Portfolio del compañero. Carteras viejas sin dueño no se reasignan automáticamente. Dos primeros accesos simultáneos pueden provocar una violación de unicidad en uno; el registro definitivo debe aprovisionar la cartera con el usuario. No hay fallback al ID 1.

Login desarrollado por el equipo

No se crearon pantallas ni tablas de usuarios. El contrato pendiente es que el login establezca el principal del contenedor y mapee USUARIO; una variable de sesión por sí sola no autoriza llamadas EJB. El equipo podrá sustituir BASIC por FORM o Jakarta Security conservando los roles. El resolver será el único punto a adaptar cuando se publique la relación definitiva con Usuario. No se tocaron ramas ajenas ni se hizo merge.

H2 y MySQL sin secretos

persistence.xml usa JTA y propiedades filtradas por Maven. El build normal apunta a ExampleDS con generación de esquema none, para no borrar tablas. demo-init utiliza create solo para H2 vacío. mysql selecciona InversorARDS y mantiene none. El driver, URL, usuario y credential-store se configuran en WildFly fuera del WAR. La plantilla CLI no contiene contraseñas. No se ejecutó ninguna migración sobre la base del compañero; se requiere revisar esquema, columna puente y conexión antes de desplegar con MySQL.

Limitaciones de la demostración

ExampleDS de H2 es temporal y no representa la base definitiva. El catálogo de la rama base se carga si está vacío; no se importan precios históricos ni datos privados. El perfil de inicialización no debe aplicarse a una base compartida. No hay lógica de migración automática ni drop-and-create. La consigna MySQL queda preparada, pero su cumplimiento en entorno real necesita despliegue y pruebas con el datasource del equipo.

06 / Validación, checklist y defensa

Verificación ejecutada

Se ejecutaron mvn test y mvn package con JDK 17 y Maven 3.9.9: diez pruebas, cero fallos y cero errores. También se construyó el perfil de demo y se desplegó en WildFly 30.0.1.Final, limitado a localhost. La prueba HTTP comprueba 401 anónimo, 403 sin rol, 400 de validación, 201 de compra y resumen correcto, aislamiento de carteras, posiciones, estado conversacional, rechazo de exceso de porcentajes, cierre y nueva conversación vacía. PRUEBAS.md conserva el detalle reproducible y la evidencia de callbacks.

Qué prueban las pruebas

JUnit/Mockito verifica reglas y cálculos; las instancias creadas con new en tests no prueban transacciones ni autorización del contenedor. Por eso se añade una prueba real sobre el WAR desplegado. Los logs del servidor muestran inicialización y destrucción de Compra y del simulador. Las pruebas de integración requieren cuentas de prueba, roles y base vacía; no se deben ejecutar sobre carteras reales. No se afirma haber probado MySQL ni el login del compañero.

Checklist de la consigna

Compra y Portfolio están implementados en capas. Se dispone de stateless y stateful con callbacks. Repository/DAO, Service Facade y Strategy están aplicados y justificados. La autenticación y el rol están activos para operaciones sensibles. Este documento tiene seis páginas. Para la entrega general aún falta integrar y demostrar Venta, conectar el login definitivo y validar MySQL. El simulador es un apoyo académico funcional y no certifica por sí solo tres componentes principales terminados.

Guion de defensa

Mostrar las interfaces y seguir una compra desde REST a JPA. Explicar por qué Portfolio dejó de estar en Compra. Calcular el ejemplo de AAPL a mano y compararlo con el JSON. Abrir dos sesiones, modificar capital en una y comprobar independencia. Cerrar y señalar el callback de destrucción. Ejecutar la misma compra sin credenciales y con rol incorrecto. Identificar en código cada uno de los tres patrones y explicar qué cambiaría al incorporar otra cotización. Finalizar mostrando qué contratos esperan a Login, BD y Venta.

Decisiones y fuentes

Se prefirió una separación pequeña y comprobable sobre nuevos frameworks. La configuración de WildFly se basa en sus guías oficiales: https://docs.wildfly.org/30/Getting_Started_Guide.html y https://docs.wildfly.org/30/WildFly_Elytron_Security.html. El comportamiento específico de este proyecto se comprobó en el servidor y las pruebas de esta rama. El checklist distingue implementación local de integración pendiente para evitar presentar trabajo ajeno como finalizado.